ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ PHẦN MỀM

MÔN HỌC: KỸ THUẬT XÂY DỰNG HỆ THỐNG AGENTIC AI - SE373

BÁO CÁO BÀI TẬP THỰC HÀNH 3

XÂY DỰNG AGENT ĐẶT VÉ MÁY BAY BẰNG LANGCHAIN VÀ ĐÁNH GIÁ BA MẪU THIẾT KẾ REACT, PLAN-THEN-EXECUTE, HYBRID

Giảng viên lý thuyết: TS. Đỗ Trọng Hợp, ThS. Ngô Ngọc Đăng Khoa, ThS. Phạm Hoàng Hải

Giảng viên thực hành: **Thầy/cô** Bùi Cao Doanh, Dương Nguyễn Phương Nam, Nguyễn Hiếu Nghĩa, Nguyễn Ngọc Quí, Nguyễn Thị Hoàng Anh, Quan Chí Khánh An

Sinh viên thực hiện: **Lê Vọng – 23521803**

Thành phố Hồ Chí Minh, Tháng 10 Năm 2026

Phần 1. TÌM HIỂU LANGCHAIN VÀ LANGGRAPH

1.1. Khái niệm và Kiến trúc LangChain

LangChain là framework mã nguồn mở phát triển ứng dụng dựa trên mô hình ngôn ngữ lớn LLM. Từ phiên bản 1.0, framework đã chuẩn hóa giao diện ổn định và định hình rõ nguyên lý cốt lõi: Agent = Model + Harness. Trong đó, Model chỉ đóng vai trò đề xuất hành động qua cơ chế tool calling, còn Harness là tầng mã nguồn lập trình truyền thống kiểm soát quyền hạn, ghi nhận quan sát, thẩm định trạng thái và ra quyết định kết thúc tác vụ.

Các thành phần tiêu biểu trong LangChain bao gồm:

• BaseChatModel: Chuẩn hóa giao tiếp native tool calling với các LLM.

• create_agent: API chuẩn thay thế AgentExecutor, tự động biên dịch thành luồng đồ thị LangGraph.

• Hệ thống Middleware: Can thiệp vòng lặp agent theo mô hình Aspect-Oriented Programming.

1.2. Hệ thống Middleware và API create_agent

API create_agent cho phép lập trình viên gắn các tầng Harness độc lập thông qua middleware mà không làm xáo trộn logic nghiệp vụ:

• **wrap_tool_call:** Cơ chế bọc ngoài mọi lời gọi công cụ. Cho phép thực thi kiểm tra quyền hạn, kiểm tra tính hợp lệ của tham số, đo lường và ngắt lời gọi vi phạm trước khi công cụ thực sự chạy.

• **ModelCallLimitMiddleware:** Cầu dao bảo vệ thiết lập ngân sách tối đa về số vòng gọi LLM, run_limit = 10, ngăn chặn hiện tượng lặp vô tận gây tiêu hao tài nguyên tính toán.

1.3. LangGraph và Cơ chế Quản lý Trạng thái

LangGraph là orchestration runtime cấp thấp cung cấp khả năng quản lý trạng thái và xây dựng đồ thị có chu trình. Mọi luồng hoạt động của agent trong LangChain thực chất được hiện thực hóa dựa trên đồ thị trạng thái LangGraph. Thông qua StateGraph, các thành phần chia sẻ chung ngữ cảnh bộ nhớ, cho phép hỗ trợ các kỹ thuật nâng cao như human-in-the-loop, checkpointing lưu vết và tua ngược trạng thái.

1.4. Mô hình GenericFakeChatModel phục vụ Testing

Trong bài tập, hệ thống sử dụng GenericFakeChatModel thuộc langchain_core.language_models. Mô hình giả lập này phát lại chính xác chuỗi thông điệp đã được kịch bản hóa, giúp hệ thống chạy độc lập hoàn toàn mà không cần API key ngoài, không tốn chi phí, và đảm bảo kết quả kiểm thử luôn tất định.

Phần 2. THIẾT KẾ HỆ THỐNG VÀ BỐN LỚP HARNESS

2.1. Mô tả Bài toán Đặt vé Máy bay

Yêu cầu của người dùng: Đặt một vé máy bay SGN đến DAD vào ngày 2026-10-07, khởi hành trước 12:00 trưa, giá vé tối đa 2.000.000 VNĐ.

Hệ thống kiểm thử trên hai bộ dữ liệu độc lập gồm trường hợp thành công và trường hợp thất bại:

Tập dữ liệu	Chuyến bay	Giờ bay	Giá vé (VNĐ)	Khởi hành sáng	Giá <= 2.000.000	Thỏa tất cả
Thành công	VN122	08:10	1.850.000	Đạt trước 12h	Đạt	HỢP LỆ
Thành công	QH118	15:40	1.640.000	Không đạt	Đạt	Vi phạm giờ bay

Thất bại	VJ604	08:10	2.480.000	Đạt trước 12h	Không đạt	Vi phạm ngân sách
Thất bại	QH118	15:40	1.640.000	Không đạt	Đạt	Vi phạm giờ bay

2.2. Hiện thực Bốn Lớp Harness

• **Lớp 1 - Ràng buộc là dữ liệu:** Yêu cầu người dùng được cấu trúc hóa thành lớp dataclass Constraints. Prompt gửi cho model được sinh động từ dữ liệu này, đồng thời phương thức is_ok dùng trực tiếp cấu trúc dữ liệu để kiểm tra tính hợp lệ khách quan. Model không thể bị trôi mục tiêu vì harness kiểm soát trực tiếp bằng mã nguồn.

• **Lớp 2 - Kiểm tra quyền hạn:** Hàm check_permission được kích hoạt ngay trước khi thực thi công cụ book_seat. Nếu chuyến bay vi phạm tiêu chí như trường hợp quá giá ở tập dữ liệu thất bại, lệnh gọi bị chặn lập tức và trả về trạng thái denied kèm lý do chi tiết.

• **Lớp 3 - Tiêu chí hoàn thành độc lập:** Hệ thống tuyệt đối không tin vào lời tuyên bố hoàn thành của LLM. Hàm is_done kiểm tra trực tiếp trạng thái hệ thống: phải tồn tại mã vé, trạng thái paid = True và toàn bộ thông số vé phải thỏa mãn Constraints.

• **Lớp 4 - Bàn giao cho con người:** Khi tác vụ thất bại, hàm handoff tổng hợp ba thông tin thiết yếu gồm: những gì đã hoàn thành, các bước đã thử kèm mã phản hồi, và một câu hỏi cụ thể giúp người dùng có thể phản hồi giải quyết trong vòng ba mươi giây.

• **Cơ chế bổ sung - Phát hiện lặp và Bịa đặt:** Tích hợp LoopDetector theo dõi ba tín hiệu gồm trùng hành động, trùng kết quả quan sát, không tiến triển mục tiêu và hàm ungrounded_facts dò tìm các số hiệu vé, giá tiền bịa đặt không có căn cứ từ công cụ.

Phần 3. HIỆN THỰC BA MẪU THIẾT KẾ AGENT

3.1. Mẫu 1: ReAct Agent

Nguyên lý: Vận hành theo chu trình lặp: Suy luận, Hành động, Quan sát. Mô hình đưa ra quyết định cho từng bước một sau khi tiếp nhận kết quả từ bước trước đó.
Cài đặt: Xây dựng bằng create_agent kết hợp middleware harness bọc ngoài.
Vết thực thi trường hợp thành công: search_flights, book_seat VN122, pay, get_booking. Kết thúc thành công sau 4 lượt gọi.
Vết thực thi trường hợp thất bại: Model thử đặt vé VJ604, bị harness chặn lập tức do vượt ngân sách. Agent dừng lại và kích hoạt bàn giao cho con người.

3.2. Mẫu 2: Plan-then-Execute Agent

Nguyên lý: Tách biệt hoàn toàn hai giai đoạn gồm Lập kế hoạch và Thực thi. Mô hình được gọi một lần duy nhất để sinh toàn bộ lộ trình qua đầu ra có cấu trúc Plan. Kế hoạch được người dùng kiểm duyệt trước khi chuyển cho mã nguồn tuần tự thực thi mà không cần gọi lại model.
Cài đặt: Xây dựng với Pydantic schema Step và biến giữ chỗ $booking_code.
Đánh đổi: Tiết kiệm chi phí gọi model, kế hoạch nhìn thấy rõ ràng trước khi chạy. Tuy nhiên, nếu một bước gặp sự cố do bị harness chặn, kế hoạch không thể tự điều chỉnh mà bắt buộc dừng cứng.

3.3. Mẫu 3: Hybrid Agent

Nguyên lý: Kết hợp ưu điểm của cả hai mô hình trên. Agent lập kế hoạch tổng thể trước khi thực thi, nhưng trong quá trình chạy, nếu một bước bị harness từ chối hoặc quan sát môi trường thay đổi đáng kể, hệ thống kích hoạt cơ chế Tái lập kế hoạch Replan. Dữ liệu các bước thất bại được gửi ngược lại cho Planner.
Cài đặt: Thiết lập giới hạn tái lập kế hoạch tối đa MAX_REPLANS = 3.
Điểm nổi bật ở trường hợp thất bại: Khi kế hoạch đầu tiên chọn VJ604 bị chặn, Hybrid không dừng ngay mà sinh kế hoạch thứ hai chọn QH118. Khi kế hoạch thứ hai tiếp tục bị từ chối do bay buổi chiều, model sinh kế hoạch thứ ba rỗng, chính thức xác nhận không có chuyến bay hợp lệ và bàn giao.

Phần 4. ĐÁNH GIÁ HIỆU QUẢ VÀ SO SÁNH

4.1. Bảng Tổng hợp Kết quả Thực nghiệm

Mẫu thiết kế	Trường hợp	Kết quả thẩm định	Số lần gọi LLM	Số lệnh Tool	Số lần Replan
ReAct	Thành công	DONE - Thành công	4 lần	4 lệnh	Không
ReAct	Thất bại	FAILED - Bàn giao	2 lần	2 lệnh	Không
Plan-then-Execute	Thành công	DONE - Thành công	2 lần	4 lệnh	Không
Plan-then-Execute	Thất bại	FAILED - Bàn giao	2 lần	2 lệnh	Không
Hybrid - Lai	Thành công	DONE - Thành công	1 lần	4 lệnh	1 lộ trình
Hybrid - Lai	Thất bại	FAILED - Bàn giao	3 lần	3 lệnh	3 lộ trình

4.2. Phân tích So sánh Chuyên sâu

1. Về chi phí và số lượng gọi mô hình:

Plan-then-Execute và Hybrid vượt trội hơn hẳn ReAct ở trường hợp thành công, chỉ tốn 1 đến 2 lần gọi LLM so với 4 lần của ReAct. ReAct tiêu tốn nhiều token nhất do lịch sử hội thoại dài thêm sau mỗi bước và chi phí token tăng nhanh theo cấp số nhân.

2. Về khả năng ứng biến và tính linh hoạt:
ReAct có tính phản xạ cao nhất ở từng bước nhỏ.
Plan-then-Execute hoàn toàn thiếu linh hoạt khi môi trường thay đổi hoặc bước chạy bị lỗi.
Hybrid là mẫu thiết kế duy nhất có khả năng thử lại thông minh, thử lần lượt các phương án khả dĩ trước khi chấp nhận thất bại, đem lại độ tin cậy cao nhất cho hệ thống.

3. Khuyến nghị ứng dụng thực tế:

Mẫu kiến trúc	Điều kiện nên lựa chọn	Rủi ro chính cần kiểm soát
ReAct	Bài toán mở, không đoán trước được số bước	Dễ lặp vô hạn, trôi dạt mục tiêu
Plan-then-Execute	Quy trình tuần tự ổn định, cần duyệt trước	Kế hoạch lỗi thời khi môi trường đổi
Hybrid - Lai	Tác vụ phức tạp, đường dài, nhiều biến động	Độ phức tạp code cao, khó gỡ lỗi hơn

Phần 5. HƯỚNG DẪN TRIỂN KHAI VÀ THỰC THI

5.1. Cấu trúc Mã nguồn Hệ thống

```
shared.py              # Định nghĩa Constraints, Mock tools, 4 lớp Harness, LoopDetector
react_agent.py         # Mẫu 1: Hiện thực ReAct Agent sử dụng LangChain create_agent
plan_execute_agent.py  # Mẫu 2: Hiện thực Plan-then-Execute Agent với Pydantic Structured Output
hybrid_agent.py        # Mẫu 3: Hiện thực Hybrid Agent kết hợp Planning và Dynamic Replanning
evaluate.py            # Kịch bản tự động hóa đo lường và đánh giá so sánh cả 3 mẫu
```

5.2. Các Lệnh Chạy Chương trình

Cài đặt thư viện phụ thuộc:

```
pip install langchain langchain-core pydantic fpdf2
```

Chạy kiểm thử từng mẫu thiết kế:

```
python react_agent.py
python plan_execute_agent.py
python hybrid_agent.py
```

Chạy kịch bản đánh giá tự động toàn diện:

```
python evaluate.py
```

Phần 6. KẾT LUẬN

1. Hiệu quả kiểm soát của Harness Engineering: Mã nguồn harness kiểm soát chặt chẽ hành vi của agent ở cả ba mẫu thiết kế. Ở trường hợp thành công, hệ thống xác nhận vé đặt thành công đúng giờ bay và ngân sách. Ở trường hợp thất bại, harness ngăn chặn hoàn toàn các hành động sai phạm như đặt chuyến bay vượt quá 2 triệu đồng hoặc bay sai buổi, đồng thời tự động kích hoạt bàn giao rõ ràng cho người dùng thay vì tin tưởng mù quáng vào phán đoán của LLM.

2. Đánh giá sự đánh đổi giữa ba mẫu kiến trúc: Thực nghiệm cho thấy không có một mẫu thiết kế nào tối ưu toàn diện trong mọi trường hợp. Plan-then-Execute tối ưu chi phí token nhưng hoàn toàn thất bại khi gặp ngoại lệ. ReAct thích nghi linh hoạt từng bước nhưng chi phí token tăng nhanh theo từng vòng lặp. Mẫu Hybrid thể hiện sự cân bằng tốt nhất giữa khả năng kiểm soát kế hoạch tổng thể và năng lực phục hồi sau sự cố thông qua cơ chế tự động lập lại kế hoạch.

3. Kết luận rút ra: Việc xây dựng hệ thống Agentic AI trong thực tế đòi hỏi sự kết hợp chặt chẽ giữa khả năng suy luận của mô hình ngôn ngữ lớn và các rào chắn kỹ thuật của tầng Harness. Mô hình chỉ nên đóng vai trò đề xuất hành động, trong khi mọi quyền kiểm tra điều kiện tiên quyết, thẩm định kết quả và quyết định dừng tác vụ bắt buộc phải do mã nguồn lập trình xác định một cách khách quan.